INSTITUT TEKNOLOGI DEL

MATERI PRAKTIKUM

Sistem Basis Data

(4142103)

D4 Teknologi Rekayasa Perangkat Lunak

Tahun akademik 2026/2027

Tanggal Sesi	:	Sesuai jadwal Minggu 3 Sesi 2
Minggu ke-/sesi	:	3/2
Topik	:	Partisi dan Ranking
Aktivitas	:	Latihan terarah, eksperimen, pengujian, dan dokumentasi.
Tujuan praktikum	:	Membedakan GROUP BY dengan window aggregate.; Menentukan partisi dan urutan yang tepat.; Memilih ranking untuk nilai seri.; Menghitung running total, moving average, LAG dan LEAD.
Waktu pengerjaan	:	120 menit
Setoran	:	Sesuai Petunjuk Praktikum
Tempat Penyerahan	:	CIS, sesuai instruksi dosen
Sarana	:	PostgreSQL, pgAdmin/Query Tool, psql/terminal bila disebut, VS Code, Git
Referensi	:	Dokumentasi resmi PostgreSQL18 sesuai daftar referensi modul

1. PENDAHULUAN DAN LUARAN

Praktikum Minggu 3 Sesi 2: Partisi dan Ranking. Kerjakan pada database minggu ini, sbd_w 03. Sesi 3 melanjutkan Sesi 2. Gunakan data fiktif dan simpan bukti hasil sendiri.

- Membedakan GROUP BY dengan window aggregate.
- Menentukan partisi dan urutan yang tepat.
- Memilih ranking untuk nilai seri.
- Menghitung running total, moving average, LAG dan LEAD.

Menit	Kegiatan
0–15	Persiapan dan checkpoint awal
15–75	Latihan terarah
75–105	Eksperimen dan matriks uji
105–120	Dokumentasi dan refleksi

Luaran
SQL, README, matriks hasil aktual, bukti/errorlog, alasan desain, dan AI Use Statement. Project menambah runbook, baseline, serta milestone.

2. PERSIAPAN DAN CHECKPOINT

1. Gunakan PostgreSQL dan pgAdmin yang dipasang pada Minggu1. Tidak perlu menginstal ulang server. VS Code dipakai untuk menyimpan SQL dan README; terminal/psql dipakai khusus perintah command-line.
2. Di pgAdmin, hubungkan server latihan sendiri, klik kanan Databases > Create > Database. Isi nama database baru sesuai minggu, pilih owner latihan yang disetujui, lalu Save. Jangan menghapus database minggu sebelumnya.
3. Klik database baru > Tools > Query Tool. Jalankan SELECT current_database(), current_user, version(); pastikan nama target benar sebelum DDL atau UPDATE.
4. Jalankan lampiran01-schema.sql,02-seed.sql,03-validasi.sql secara berurutan. Dapat diketik dari lampiran atau dibuka dari folder SQL pendamping. Pilih hanya statement yang sedang dikerjakan lalu Execute/F5.
5. Simpan script latihan dengan nomor langkah, README, dan folder bukti. Catat hasil query pada Data Output dan pesan pada Messages. Screenshot harus memperlihatkan konteks database serta statement, tanpa password.
6. Sesi3 meneruskan database minggu yang sama dari Sesi2. Jangan mengulang DDL yang tidak idempotent. Setelah error di transaksi eksplisit jalankan ROLLBACK, kemudian perbaiki penyebab.
7. Untuk dua koneksi, buka dua Query Tool terpisah ke database sama. SELECT pg_backend_pid() harus berbeda. Jalankan langkah A/B bergantian menurut komentar, bukan seluruh blok sekaligus.
8. Baris # adalah perintah terminal atau konfigurasi contoh. Pseudocode dan struktur folder bukan SQL. Perintah COPY server berbeda dari \copy psql; baca petunjuk konteks sebelum eksekusi.

```
sbd-week-03/
  sesi-02/
    latihan.sql
    README.md
    bukti/

-- Query Tool pada database sbd_w03:
SELECT current_database(),current_user,pg_backend_pid();
SELECT COUNT(*),SUM(total) FROM lab.pesanan;
```

Checkpoint
Databasebenar; seedbaru5pesanan,total950000;4produk,totalstok43. Jika meneruskanSesi2, gunakan keadaanakhir yang dicatat. Janganresetdatabersama.

3. OVER DAN IDENTITAS BARIS

GROUP BY mengubah grain menjadi satu baris per kelompok. Window function menghitung nilai lintas baris sambil mempertahankan identitas baris sumber. OVER menandai fungsi sebagai window. Rata-rata pelanggan dapat muncul pada setiap pesanan pelanggan itu. Window bekerja terhadap baris yang lolos tahap sebelumnya. Karena itu WHERE status selesai memengaruhi data yang dihitung. Tentukan populasi laporan sebelum melihat hasil statistik.

Langkah pengerjaan

1. Pastikan konteks eksekusi: SQL di Query Tool/psql, command-line di terminal, dan pseudocode hanya dibaca.
2. Simpan potongan berikut pada latihan.sql atau file terminal yang sesuai. Untuk A/B jalankan bergantian; jangan Execute seluruh blok.
3. Bandingkan dengan AVG GROUP BY.
4. Periksa Ana pada dua baris.
5. Hitung rata-rata secara manual.
6. Catat hasil aktual, jumlah baris, pesan error, dan alasan. Jika transaksi error, rollback koneksi tersebut sebelum melanjutkan.

```
SELECT id,pelanggan_id,total,
 AVG(total) OVER(PARTITION BY pelanggan_id) rata
FROM lab.pesanan WHERE pelanggan_id IS NOT NULL
ORDER BY id;
```

Penjelasan sintaks yang perlu dipahami

OVER: fungsi dihitung pada jendela baris tanpa meruntuhkan baris seperti GROUP BY. PARTITION BY membagi kelompok, ORDER BY di dalam OVER menentukan urutan perhitungan, sedangkan ORDER BY paling luar menentukan urutan hasil yang di tampilkan.

SELECT: baca kolom hasil terlebih dahulu, lalu FROM sebagai sumber data. Alias memberi nama singkat atau peran tabel. WHERE menyaring baris, sedangkan ORDER BY menentukan urutan keluaran. Nilai COUNT dan SUM harus ditafsirkan sesuai grain hasil, bukan hanya dilihat sebagai angka.

> Hasil yang diharapkan
> 4 baris tetap ada, rata Ana 100000.

Laporan pengamatan

Isi: prediksi; tindakan; hasil aktual; bukti; perbedaan; perbaikan; retest. Untuk kinerja tulis metode dan median aktual. Untuk konkurensi tulis urutanA/B serta PID. Harapan modul tidak boleh disalin sebagai hasil aktual tanpa pengujian.

4. PARTITION BY DAN ORDER BY

Partisi memisahkan kelompok perhitungan. ORDER BY di dalam OVER menentukan urutan analitik, sedangkan ORDER BY akhir mengurutkan tampilan. Kedua urutan tidak otomatis sama. Baris dengan nilai seri membutuhkan tie breaker untuk nomor yang stabil. Pada nomor pesanan gunakan tanggal lalu ID. Pada peringkat nilai gunakan total saja jika nilai sama harus mempunyai rank yang sama. Menambahkan ID pada ranking akan menghilangkan makna seri.

Langkah pengerjaan

1. Pastikan konteks eksekusi: SQL di Query Tool/psql, command-line di terminal, dan pseudocode hanya dibaca.
2. Simpan potongan berikut pada latihan.sql atau file terminal yang sesuai. Untuk A/B jalankan bergantian; jangan Execute seluruh blok.
3. Uji urutan tanggal lalu ID.
4. Ubah ORDER BY akhir tanpa mengubah OVER.
5. Periksa nomor reset saat pelanggan berubah.
6. Catat hasil aktual, jumlah baris, pesan error, dan alasan. Jika transaksi error, rollback koneksi tersebut sebelum melanjutkan.

```
SELECT id,pelanggan_id,
 ROW_NUMBER() OVER(PARTITION BY pelanggan_id
 ORDER BY tanggal,id) nomor
FROM lab.pesanan WHERE pelanggan_id IS NOT NULL
ORDER BY pelanggan_id,tanggal,id;
```

Penjelasan sintaks yang perlu dipahami

OVER: fungsi dihitung pada jendela baris tanpa meruntuhkan baris seperti GROUP BY. PARTITION BY membagi kelompok, ORDER BY di dalam OVER menentukan urutan perhitungan, sedangkan ORDER BY paling luar menentukan urutan hasil yang di tampilkan.

SELECT: baca kolom hasil terlebih dahulu, lalu FROM sebagai sumber data. Alias memberi nama singkat atau peran tabel. WHERE menyaring baris, sedangkan ORDER BY menentukan urutan keluaran. Nilai COUNT dan SUM harus ditafsirkan sesuai grain hasil, bukan hanya dilihat sebagai angka.

> Hasil yang diharapkan
> Nomor Ana 1,2. Budi dan Citra mulai 1.

Laporan pengamatan

Isi: prediksi; tindakan; hasil aktual; bukti; perbedaan; perbaikan; retest. Untuk kinerja tulis metode dan median aktual. Untuk konkurensi tulis urutanA/B serta PID. Harapan modul tidak boleh disalin sebagai hasil aktual tanpa pengujian.

5. ROW_NUMBER, RANK, DAN DENSE_RANK

ROW_NUMBER memberi nomor unik sesuai urutan. RANK memberi peringkat sama pada nilai seri lalu melompati posisi berikutnya. DENSE_RANK tidak melompati posisi. Pada total 500000,200000,100000,100000,50000 rank menjadi 1,2,3,3,5. Dense rank menjadi 1,2,3,3,4. Gunakan tie breaker untuk ROW_NUMBER tetapi jangan menambahkan ID ke RANK bila seri berdasarkan total harus dipertahankan.

Langkah pengerjaan

1. Pastikan konteks eksekusi: SQL di Query Tool/psql, command-line di terminal, dan pseudocode hanya dibaca.
2. Simpan potongan berikut pada latihan.sql atau file terminal yang sesuai. Untuk A/B jalankan bergantian; jangan Execute seluruh blok.
3. Tulis hasil prediksi lima baris.
4. Bandingkan tiga fungsi pada total 100000.
5. Jelaskan pilihan fungsi untuk laporan top nilai.
6. Catat hasil aktual, jumlah baris, pesan error, dan alasan. Jika transaksi error, rollback koneksi tersebut sebelum melanjutkan.

```
SELECT id,total,
 ROW_NUMBER() OVER(ORDER BY total DESC,id) nomor,
 RANK() OVER(ORDER BY total DESC) peringkat,
 DENSE_RANK() OVER(ORDER BY total DESC) padat
FROM lab.pesanan ORDER BY total DESC,id;
```

Penjelasan sintaks yang perlu dipahami

OVER: fungsi dihitung pada jendela baris tanpa meruntuhkan baris seperti GROUP BY. PARTITION BY membagi kelompok, ORDER BY di dalam OVER menentukan urutan perhitungan, sedangkan ORDER BY paling luar menentukan urutan hasil yang di tampilkan.

SELECT: baca kolom hasil terlebih dahulu, lalu FROM sebagai sumber data. Alias memberi nama singkat atau peran tabel. WHERE menyaring baris, sedangkan ORDER BY menentukan urutan keluaran. Nilai COUNT dan SUM harus ditafsirkan sesuai grain hasil, bukan hanya dilihat sebagai angka.

> Hasil yang diharapkan
> Rank terakhir 5, dense rank terakhir 4.

Laporan pengamatan

Isi: prediksi; tindakan; hasil aktual; bukti; perbedaan; perbaikan; retest. Untuk kinerja tulis metode dan median aktual. Untuk konkurensi tulis urutanA/B serta PID. Harapan modul tidak boleh disalin sebagai hasil aktual tanpa pengujian.

6. TOP-N MELALUI CTE

Window tidak dapat langsung digunakan pada WHERE di level SELECT yang sama. Gunakan CTE atau subquery: hitung peringkat dahulu lalu saring. Top dua baris berbeda dari top dua nilai. ROW_NUMBER membatasi jumlah baris, sedangkan DENSE_RANK dapat mempertahankan semua seri sehingga jumlah hasil lebih banyak. Tulis kebutuhan dengan jelas agar laporan tidak berubah ketika terdapat seri.

Langkah pengerjaan

1. Pastikan konteks eksekusi: SQL di Query Tool/psql, command-line di terminal, dan pseudocode hanya dibaca.
2. Simpan potongan berikut pada latihan.sql atau file terminal yang sesuai. Untuk A/B jalankan bergantian; jangan Execute seluruh blok.
3. Uji filter CTE.
4. Bandingkan rank<=3 dengan row_number<=3.
5. Catat jumlah hasil saat seri masuk batas.
6. Catat hasil aktual, jumlah baris, pesan error, dan alasan. Jika transaksi error, rollback koneksi tersebut sebelum melanjutkan.

```
WITH x AS (
 SELECT *,DENSE_RANK() OVER(ORDER BY total DESC) r
 FROM lab.pesanan)
SELECT id,total FROM x WHERE r<=2 ORDER BY total DESC,id;
```

Penjelasan sintaks yang perlu dipahami

OVER: fungsi dihitung pada jendela baris tanpa meruntuhkan baris seperti GROUP BY. PARTITION BY membagi kelompok, ORDER BY di dalam OVER menentukan urutan perhitungan, sedangkan ORDER BY paling luar menentukan urutan hasil yang di tampilkan.

SELECT: baca kolom hasil terlebih dahulu, lalu FROM sebagai sumber data. Alias memberi nama singkat atau peran tabel. WHERE menyaring baris, sedangkan ORDER BY menentukan urutan keluaran. Nilai COUNT dan SUM harus ditafsirkan sesuai grain hasil, bukan hanya dilihat sebagai angka.

> Hasil yang diharapkan
> Top dua nilai menghasilkan id 5 dan id 3.

Laporan pengamatan

Isi: prediksi; tindakan; hasil aktual; bukti; perbedaan; perbaikan; retest. Untuk kinerja tulis metode dan median aktual. Untuk konkurensi tulis urutanA/B serta PID. Harapan modul tidak boleh disalin sebagai hasil aktual tanpa pengujian.

7. LATIHAN MANDIRI DAN MATRIKS UJI

1. Jalankan kembali kasusnormal tanpa melihat hasilteman.
2. Tambahkan1kasusbatas atauinvalid yang relevan.
3. Nyatakan prediksi sebelum eksekusi.
4. Cataterror danstatus transaksi; perbaiki lalu retest.
5. Jelaskan1keputusan teknis serta1trade-off.

Kasus	Harapan
Jumlah output	5 untuk ranking global
Rank seri	id 1/id 2 rank 3
Dense rank akhir	4
Top 2	id 5/id 3

Format yang diisi mahasiswa

Kondisi/input	Tindakan	Aktual	Status/bukti
Normal	Diisi	Setelah uji	Diisi
Batas/invalid	Diisi	Setelah uji	Diisi
Regresi	Diisi	Setelah uji	Diisi

Penilaianbukti
Screenshot tanpa statement dan konteks belum cukup. Bagian belum diuji harus ditandai belum diuji, bukan lulus.

8. TROUBLESHOOTING DAN PENGGUNAAN AI

Gejala	Pemeriksaan
Connection refused	Periksa service PostgreSQL, host localhost, port, dan server pgAdmin; jangan ubah server kampus.
Password/permission error	Gunakan akun yang disetujui; jangan memasukkan password ke kode. Role latihan dapat disiapkan dosen.
Relation/column does not exist	Periksa current_database(), schema lab, urutan seed/migration, dan nama objek.
Duplicate object/key	Periksa objek/ledger; jangan mengulang DDL sekali-jalan. Seed tidak mereset nilai yang sudah berubah.
Current transaction is aborted	ROLLBACK pada koneksi yang gagal, lalu ulangi transaksi dengan penyebab diperbaiki.
Query menunggu	Periksa transaksi A/B, pg_blocking_pids, timeout sesi, dan lepaskan transaksi sendiri.
Plan berbeda dari contoh	Periksa row count, statistics, index, versi, cache, dan selectivity; plan tidak dijamin identik.
pg_dump/psql tidak dikenali	Gunakan SQL Shell atau tambahkan folder bin instalasi PostgreSQL sesuai komputer; jangan menaruh password di command line.

- AI hanya dipakai dengan datafiktif untuk penjelasan/diagnosis.
- Jangan memasukkan password, token, connectionstringrahasia, atau data pribadi.
- Periksa saran AI terhadap dokumentasi resmi dan tes aktual.
- Catat alat, tujuan, bagian bantuan, verifikasi, serta koreksi; jika tidakmemakaiAI nyatakan tidakmenggunakan.
- Mahasiswa harus dapat menjelaskan SQL sendiri saat walkthrough.

9. SETORAN DAN PERSIAPAN BERIKUTNYA

- Script SQL dan file pendukung sesuai urutan.
- README: versi, DBtarget, caraeksekusi, kondisi awal/akhir, hasiluji, dan batasan.
- Bukti yang aman: hasilquery, errorlog, plan, dan pengukuran bila relevan.
- AI Use Statement dan sumberreferensi.
- Pada project: proposal/ERD, baseline, securitychecklist, migration/test, recoveryrunbook, dan bukti milestone yang relevan.

```
# Namaarsipusulan:
sbd_4142103_w03s02_NIM.zip
# Jangan masukkan credential, database nyata, atau filesementara.
```

Penyerahan mengikuti pengumuman dosen pada CIS. Modul ini tidak menetapkan bataspenyerahan baru. Periksa seluruh isi arsip sebelum menyerahkan.

> Tugas persiapan
> Siapkan tabel prediksi akumulasi untuk Sesi 3.

LAMPIRAN: 01-schema.sql

Untuk persiapan Sesi 2 pada database baru. Sesi 3 tidak perlu mengulang seed jika keadaan akhir Sesi 2 sudah sesuai.

```
CREATE SCHEMA IF NOT EXISTS lab;
CREATE TABLE IF NOT EXISTS lab.pelanggan (
 id integer PRIMARY KEY, nama text NOT NULL,
 mentor_id integer REFERENCES lab.pelanggan(id));
CREATE TABLE IF NOT EXISTS lab.produk (
 id integer PRIMARY KEY, nama text NOT NULL,
 harga numeric(12,2) NOT NULL CHECK(harga >= 0),
 stok integer NOT NULL CHECK(stok >= 0),
 versi integer NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS lab.pesanan (
 id integer PRIMARY KEY,
 pelanggan_id integer REFERENCES lab.pelanggan(id),
 tanggal date NOT NULL, total numeric(12,2) NOT NULL,
 status text NOT NULL CHECK(status IN ('SELESAI','PENDING')));
```

LAMPIRAN: 02-seed.sql

Untuk persiapan Sesi 2 pada database baru. Sesi 3 tidak perlu mengulang seed jika keadaan akhir Sesi 2 sudah sesuai.

```
INSERT INTO lab.pelanggan VALUES
 (1,'Ana',NULL),(2,'Budi',1),(3,'Citra',1),(4,'Dedi',2)
ON CONFLICT DO NOTHING;
INSERT INTO lab.produk(id,nama,harga,stok) VALUES
 (1,'Buku',20000,10),(2,'Mouse',100000,20),
 (3,'SSD',500000,5),(4,'Keyboard',200000,8)
ON CONFLICT DO NOTHING;
INSERT INTO lab.pesanan VALUES
 (1,1,'2026-09-01',100000,'SELESAI'),
 (2,1,'2026-09-02',100000,'SELESAI'),
 (3,2,'2026-09-02',200000,'SELESAI'),
 (4,NULL,'2026-09-03',50000,'PENDING'),
 (5,3,'2026-09-04',500000,'SELESAI')
ON CONFLICT DO NOTHING;
```

LAMPIRAN: 03-validasi.sql

Untuk persiapan Sesi 2 pada database baru. Sesi 3 tidak perlu mengulang seed jika keadaan akhir Sesi 2 sudah sesuai.

```
SELECT current_database(), current_user, version();
SELECT count(*) FROM lab.pelanggan; -- 4
SELECT count(*) FROM lab.produk; -- 4
SELECT count(*),sum(total) FROM lab.pesanan;
-- 5 baris, total 950000
SELECT * FROM lab.pesanan ORDER BY id;
```

REFERENSI

PostgreSQL 18 - Window function tutorial: https://www.postgresql.org/docs/18/tutorial-window.html
PostgreSQL 18 - Window functions: https://www.postgresql.org/docs/18/functions-window.html
PostgreSQL 18 - Window syntax and frames: https://www.postgresql.org/docs/18/sql-expressions.html